

Debugging

- systematic way of fixing your code
- more powerful than `print()`

| the hardest bugs to find are bugs without error

Definitions

Breakpoint

- a point in your code you define to stop the execution at
-

Print debugging

- scatter `print` messages throughout your program
- surprisingly effective

Warning

`print` debugging will fail for several errors.
especially errors without error message are very hard to find.

Where `print` can fail

- [Race conditions](#)
 - [Async scheduling](#)
 - [State changing between prints](#)
 - [Deadlocks](#)
 - [Mutable object aliasing](#)
 - [Swallowed exceptions](#)
 - [Memory corruption](#)
 - [Infinite loop](#)
-

Where Debuggers can help

- Heisenbugs
 - Mutable object aliasing
 - Swallowed exceptions
 - Memory corruption
 - Infinite loop
 - complex/incorrect variable states
-

Debuggers

Python

pdb

- <https://docs.python.org/3/library/pdb.html>
- **THE** python debugger

Running the debugger

```
python -m pdb <script.py>    #standard python
uv run -m pdb <script.py>    #using uv
```

- add **#Breakpoint** via builtin `breakpoint()` function
- useful commands
 - `s(tep)`
 - execute current line
 - stop at first possible occasion
 - function call
 - next line
 - `n(ext)`
 - continue execution until next line in **current** function
 - does **not** step **into** functions
 - `a(rgs)`
 - ☐ print args of current function
 - show current values of args
 - `l(ist)`
 - list 11 lines around the current line
 - `h(elp) [command]`
 - list of available commands
 - description of `command` (if `command` is passed)

- `w(here)`
 - show stack trace
- `d(own) [count]`
 - move `count` frames down in the stack
- `u(p) [count]`
 - move `count` frames up in the stack
- `b(reak) [lineno]`
 - add breakpoint to `lineno` in current file
 - list all breakpoints (if no argument)
- `unt(il) [lineno]`
 - execute until `lineno` is reached
- `c(ontinue)`
 - continue until next breakpoint
- `j(ump) lineno`
 - set line that will be executed next
 - allows to
 - rerun code
 - skip code
- `p expression`
 - evaluate `expression` in current context
 - print value of `expression`
- `pp expression`
 - pretty print value of `expression`
- `restart`
 - restart debugged [python](#) program
- `q(uit)`
 - quit debugging
- inspecting the state
 - `pp locals()` ... show local variables
 - `pp globals()` ... show global variables
 - `pp [(k, id(v), v) for k, v in locals().items()]` ... show locals variables with additional info

Julia

- <https://github.com/JuliaDebug/Debugger.jl>
- currently (20260708) official [Julia](#) debugger

Debugger.jl

Running the debugger

```
julia --project=. -i    #launch interactive julia session
```

```
using Debugger
include(<script.jl>)
@enter <function to debug> #enters `function` from first line
@run <function to debug>   #enters `function` and runs until first
breakpoint
```

Note

it might be useful to define a `main()` function that contains all your logic so Julia does not need to execute your script and you can debug the whole thing

- add `#Breakpoint` via `@bp`
 - see [breakpoint docs](#) for more info
- useful commands
 - `q`
 - quit debugger
 - `C`
 - toggle compiled mode
 - `n`
 - step to next line
 - `s`
 - step into next call
 - `so`
 - step out of current call
 - `c`
 - continue until breakpoint
 - `nc`
 - step to next call
 - `se`
 - step one expression step
 - `si`
 - step into call if next expression is a call
 - `st`
 - show status
 - `fr i Int`
 - show all variables in frame `i`
 - show all variables in current frame (if no arg)
 - `bp`

- show all breakpoints
- bp add line
 - add breakpoint to line